Initial Project Plan

Contents

1 Project Overview **2**

2 Dependencies **2**

3 Project Milestones (Overall) **2**

4 All Sub-Teams Must Complete **2**
- 4.1 1. Sub-Team Class Diagram . . . 2
- 4.2 2. Use Case Diagram + Descriptions . . . 3
- 4.3 3. State Machine Diagram . . . 3
- 4.4 4. Design Presentation . . . 4
- 4.5 5. Planning Document . . . 4

5 Sub-Team Specific Focus Areas **5**
- 5.1 Platform Core Sub-Team . . . 5
- 5.2 Client/UI Sub-Team . . . 7
- 5.3 Rules + Validation Sub-Team . . . 9
- 5.4 Quality + Testing Sub-Team . . . 11
- 5.5 Integration + Release Sub-Team . . . 12

6 Submission Checklist **13**

7 Individual Requirements **14**

8 Key Reminders & Principles **14**

9 Project Milestones (Overall) **15**

10 Timeline Recommendations **15**

11 Quality Checklist **16**

12 Tools & Resources **17**

13 Common Pitfalls to Avoid **17**

14 Success Criteria **18**

1 Project Overview

Platform: Gaming Platform with Client UI (React/TypeScript) and Platform Core (MongoDB/Django).

2 Dependencies

- Client UI: React, TypeScript
- Platform Core: MongoDB, Django (Python with classes)

3 Project Milestones (Overall)

- Having a game (end-to-end functionality)
- Create account
- Log in
- View games list
- Chat functionality
- Create game
- Join game
- Leaving a game
- View dashboard
- Player status
- Testing milestones (to be defined by Quality team)

4 All Sub-Teams Must Complete

4.1 1. Sub-Team Class Diagram

Deliverable: `class_diagram_[subteam].png` or `class_diagram_[subteam].svg`

Tasks

- Identify all core classes within your sub-team's area of responsibility
- Define attributes for each class
- Define methods for each class
- Map relationships between classes (inheritance, composition, aggregation, association)
- Identify and document interface points with other sub-teams' components
- Create visual diagram using UML notation
- Export as PNG or SVG format
- Review with team for completeness and accuracy

Requirements

- Must show **your** sub-team's classes only
- Must include clear interface points to other sub-teams
- Must use proper UML notation

- Format: PNG or SVG only

4.2 2. Use Case Diagram + Descriptions

Deliverable: `use_cases_[subteam].pdf`

Tasks

- Identify 2–4 key use cases your sub-team is responsible for
- Create use case diagram showing:
  - All actors involved
  - System boundary
  - Use case interactions
  - Relationships (includes, extends, generalizations)
- Write detailed descriptions for **each** use case including:
  - Actors
  - Preconditions
  - Main flow
  - Postconditions
  - Exceptions
- Format all content into a single PDF
- Review descriptions for clarity and completeness

Requirements

- Minimum 2 use cases, maximum 4
- Each use case must have complete documentation
- Must show interaction with system
- Format: PDF only

4.3 3. State Machine Diagram

Deliverable: `state_machine_[subteam].png` or `state_machine_[subteam].svg`

Tasks

- Identify at least one important lifecycle/flow your sub-team manages
- Define all possible states
- Define all transitions between states
- Label transitions with triggers/events
- Identify initial and final states
- Document any guard conditions
- Create visual state machine diagram
- Export as PNG or SVG format
- Validate completeness of state coverage

Examples by Sub-Team

- Platform Core: Lobby lifecycle, game session states
- Client/UI: User session states, screen navigation flow
- Rules + Validation: Validation flow, move processing states
- Quality + Testing: Test execution lifecycle
- Integration + Release: Build/deployment pipeline states

Requirements

- Minimum one state machine diagram
- Must represent important lifecycle in your domain
- Format: PNG or SVG only

4.4 4. Design Presentation

Deliverable: Presentation in tutorial session

Tasks

- Create presentation slides/materials
- Prepare content covering:
  - The problem: what challenge your sub-team is addressing
  - Design solution: your proposed architecture/approach
  - Integration points: how your design connects with other sub-teams
  - Assumptions: what you're assuming to be true
  - Trade-offs: design decisions and their implications
- Practice presentation as a team
- Ensure all team members can answer questions about any part
- Prepare for Q&A from instructors and other teams
- Time the presentation appropriately

Requirements

- Every team member must be prepared to present
- Every team member must be able to answer questions
- Must cover all required topics
- Be concise and clear

4.5 5. Planning Document

Deliverable: `planning_[subteam].pdf`

Tasks

- List your sub-team's milestones for P2 (current phase)
- List your sub-team's milestones for P3 (implementation phase)
- Identify dependencies on other sub-teams:
  - What you need from Platform Core
  - What you need from Client/UI

  - What you need from Rules + Validation
  - What you need from Quality + Testing
  - What you need from Integration + Release
- Create high-level task breakdown for implementation phase
- Estimate effort/timeline for major tasks
- Identify risks and mitigation strategies
- Format into organized PDF document

Requirements

- Must include P2 and P3 milestones
- Must explicitly list inter-team dependencies
- Must provide implementation task breakdown
- Format: PDF only

5 Sub-Team Specific Focus Areas

5.1 Platform Core Sub-Team

Primary Design Focus

- Room/lobby lifecycle design
- Turn progression system
- Game state management architecture
- Core data models (User, Game, Room, etc.)
- Database schema design (MongoDB)
- Django backend API structure

Key Deliverables

- Class diagram showing data models and core services
- Use cases: create lobby, join game, progress turn, etc.
- State machine: lobby lifecycle or game session states
- Planning: database setup, API endpoints, data persistence

Milestones

- Finalize Platform Core scope and ownership: lobby lifecycle, session management, turn progression, persistence boundaries
- Finalize core data models (Game, Player, Move, WebSocket message schema) and event contracts (event, payload, timestamp)
- Finalize API/WebSocket design: REST for create/join/leave/fetch-state and WebSocket `/ws/game/{game_id}` for live gameplay
- Finalize Iteration 1 deliverables (class diagram, use cases, state machine, planning + dependencies + risks)

- Implement lobby lifecycle features (public/private lobby, 4-letter code, host controls, max-player cap)
- Implement server-authoritative turn/game-state loop with WebSocket room/session management
- Implement MongoDB persistence for state, turn, status, and move history; support reconnect/resync
- Integrate with Rules + Validation, Client/UI, and Quality + Testing for first end-to-end playable flows
- Harden reliability/security (disconnect handling, duplicate/late message handling, token checks, conflict/version control)
- Resolve edge cases (host leaves, empty lobby, in-progress disconnects, reconnect correctness)
- Stabilize event/API contracts and documentation for final release/demo
- Support final integrated showcase with stable lobby → game → turn → game-over flows

Dependencies

- REST endpoints for create/join/leave/fetch-state flows
- WebSocket event contract used by all teams (`join_game`, `make_move`, `reconnect`, `game_state`, `turn_changed`, `move_rejected`, etc.)
- Rules + Validation dependency for legal move decisions and rejection reasons before state mutation/turn advance
- Client/UI dependency on stable event formats and reconnect flow behavior
- Quality + Testing dependency on deterministic flow contracts for E2E scenarios
- Integration + Release dependency for shared stack build/run process, CI integration, and release documentation
- Auth/session token validation dependency for secure WebSocket connection acceptance and action authorization

High-Level Task List

- Define and document lobby lifecycle states and transitions (create/join/leave/start)
- Define core data models and ownership boundaries for Game, Player, Move, and session membership
- Define REST + WebSocket interface contracts and standardized event payload format
- Define reconnect/resync design (fetch latest state, rejoin room, conflict handling)
- Produce Iteration 1 artifacts and cross-team dependency map
- Build lobby REST endpoints and host moderation actions
- Build WebSocket connection manager (`game_id` → sockets) and room routing
- Implement turn progression/game-state update loop on server side
- Persist/load game state and move history via MongoDB
- Implement validation/error flow integration (`move_accepted`/`move_rejected`) with no invalid state mutation
- Implement disconnect/reconnect and stale-socket cleanup logic
- Add robustness features: version checks, duplicate/late message handling, timeout behavior
- Finalize edge-case handling and stabilize inter-team contracts

- Support full integration testing and demo-script reliability
- Finalize technical docs and handoff notes for release/demo

5.2 Client/UI Sub-Team

Primary Design Focus

- Screen flow design and navigation
- UI mockups for key screens
- Shared React components library
- User interaction design patterns
- State management approach (Redux, Context, etc.)
- Component hierarchy

Key Deliverables

- Class diagram showing component structure and state management
- Use cases: navigate screens, view dashboard, interact with UI, etc.
- State machine: user session states or screen navigation flow
- Planning: component development, styling approach, API integration

Additional Tasks

- Create wireframes/mockups for major screens
- Define UI component naming conventions
- Establish styling/theming approach

Milestones

- Initial setup & design alignment
- Project scaffolding (React/TypeScript boilerplate, folder structure, routing)
- Define state management approach (Redux, Context, etc.)
- Establish shared component library structure and styling/theming conventions
- Wireframes/mockups for all major screens (welcome, login, registration, dashboard, lobby, game, chat)
- Core screens & navigation:
  - Welcome/landing page with game list display (UC-01)
  - Registration form with input validation and error handling (UC-02)
  - Login form with credential submission and error states (UC-03)
  - Screen navigation flow with route guards (authenticated vs. unauthenticated)
- Authenticated experience:
  - Dashboard view with dynamic user status display (idle, in lobby, in queue, game ready) (UC-04)
  - Lobby UI showing participants and lobby selection/creation (UC-05)
  - Queue UI with status indicators and leave-queue option (UC-06)
  - Chat interface with lobby and global channel contexts (UC-07)

- API integration & polish:
  - Connect all screens to Platform Core API endpoints (auth, lobby, queue, chat)
  - Real-time updates for chat messages, queue status, and game-ready notifications
  - Error handling and loading states across all views
  - Work with Quality + Testing sub-team to ensure frontend test coverage

Dependencies

From Platform Core

- API endpoints
- Data model contracts
- Core services (room/lobby, game/session state, turn progression)

From Rules + Validation

- Validation interface
- Rule definitions
- Validation/error response format (invalid move/error states)

From Quality + Testing

- Test utilities
- Mock data
- Testing guidance (unit/integration/E2E expectations)

From Integration + Release

- CI/CD pipeline
- Build scripts
- Documentation standards

External/stub integration points UI must connect to

- User authentication database (stubbed interface)
- Game data storage (match history/leaderboard, stubbed interface)
- Hosting/network interface (stubbed interface)

High-Level Task List

- Project setup and architecture:
  - Scaffold React/TypeScript project and configure routing
  - Define global state management (auth, lobby, queue, chat, game)
  - Establish shared component library and styling conventions
  - Configure API client and environment setup
- Core screens and navigation:
  - Implement welcome, registration, and login screens
  - Build dashboard with dynamic user status display
  - Implement lobby creation/join/leave UI

  - Implement queue interface with status indicators
  - Implement chat interface
- Game screen and validation integration:
  - Build game screen and render server state
  - Implement move submission to Rules + Validation endpoint
  - Handle validation responses (valid/invalid move states)
  - Display turn indicators and end-of-game states
- API integration and real-time updates:
  - Connect all views to Platform Core endpoints
  - Implement real-time updates for chat, queue, and game state
  - Integrate stubbed authentication and game storage interfaces
- Error handling, testing, and release:
  - Standardize error handling and loading states
  - Work with Quality + Testing team for frontend coverage
  - Align with CI/CD pipeline and documentation standards

5.3 Rules + Validation Sub-Team

Primary Design Focus

- Move validation pipeline architecture
- Game-agnostic validation interfaces
- Error state handling and messaging
- Validation rule engine design
- Turn validation workflow
- Rule extensibility framework

Key Deliverables

- Class diagram showing validation pipeline and rule interfaces
- Use cases: validate move, handle invalid input, process turn, etc.
- State machine: validation flow or move processing states
- Planning: validation framework, rule definitions, error handling

Additional Tasks

- Define validation error taxonomy
- Design rule configuration format
- Plan for game-specific rule plugins

Milestones

- Initial alignment:
  - Move state created
  - Game state created
  - Error state created

  - Creation of service layer classes
- Basic end-to-end structure:
  - Define endpoints to accept moves
  - Define endpoints to validate and apply move
  - Define pipelines to send validation back to client
  - Define endpoint for user creation/login
- Playable game:
  - Full implementation of pipeline for move validation
  - Work with test subgroup to create proper test cases
  - Properly storing user stats

Dependencies

Validation Dependencies

- Win conditions, legal moves, turn order, tie rules, edge cases
- Validation results (isValid, error, etc.)

Authorization Dependencies

- Which player is making the move
- Which player belongs to which game room
- Whose turn it is

Game Dependencies

- Game state (lobby, ready, in progress, finished)
- Start conditions (minimum players, players ready)

Error Handling Dependencies

- Validate move, turn, room, game status
- Global exception handling

Storage Dependencies

- Finished game results (win/loss)

Testing Dependencies

- Edge cases (illegal moves, boundary conditions, simultaneous requests, reconnection)

High-Level Task Breakdown

- Validation:
  - Create endpoint logic for user validation, creation, and deletion
  - Endpoint logic for proper move validation
  - Standardize validation error codes
  - Ensure validation is deterministic
- Authorization:

  - Implement logic to map user → room → player slot
  - Enforce who can act
  - Implement "whose turn" checks
  - Add game status restrictions (can't move if game is finished or if in lobby, etc.)
- Game lifecycle:
  - Implement start conditions (minimum players, players ready)
  - Start game and finish game conditions (what triggers finish)
  - Implement reconnection/resync behavior
- Error handling:
  - Proper handling of errors with games, user login, creation, and deletion
  - Add validation for missing game, missing room, invalid state transitions
  - Handle simultaneous requests
- Storage:
  - Save on finish workflow and update user stats
  - Implement retrieval endpoints: fetch user stats, fetch game history, fetch summary, etc.
- Testing:
  - Create testing matrix for each rule category: illegal/legal moves, boundaries, win/tie, turn order
  - Add integration tests for endpoints (accept move → validate/apply → response pipeline)
  - Coordinate with test subgroup to convert edge cases into automated tests + expected outputs

5.4 Quality + Testing Sub-Team

Primary Design Focus

- Overall test strategy (unit, integration, E2E)
- Integration test architecture
- Demo script framework
- Test data management
- Continuous testing approach
- Quality metrics and reporting

Key Deliverables

- Class diagram showing test framework and utilities
- Use cases: run test suite, execute integration test, generate demo, etc.
- State machine: test execution lifecycle
- Planning: test coverage goals, testing tools, automation strategy

Additional Tasks

- Define test coverage requirements per sub-team
- Select testing frameworks (Jest, Pytest, etc.)
- Design demo script structure
- Plan performance/load testing approach

Milestones

- Define test coverage requirements per sub-team
- Finalize unit/integration/E2E strategy + ownership
- Select and configure frameworks (Pytest)
- Set up backend & frontend test structures (baseline coverage)
- Implement unit tests (backend + frontend)
- Implement API integration tests
- Implement frontend–backend flow tests
- Implement core gameplay E2E scenarios
- Design demo script framework + test data strategy
- Build integration test architecture

Dependencies

- Stable API contracts and endpoints from all sub-teams
- Game state and validation pipeline finalized
- Feature-complete staging build for demo script execution

High-Level Task Breakdown

- Define unit, integration, and E2E test strategy and ownership per sub-team
- Select and configure testing frameworks (Jest, Pytest, Playwright/Cypress)
- Design test framework class diagram with utilities and data factories
- Build state machine for test execution lifecycle (queued $\rightarrow$ running $\rightarrow$ passed/failed $\rightarrow$ reported)
- Design test data seeding, isolation, and teardown patterns
- Build demo script structure with repeatable seed, run, and teardown flow

5.5 Integration + Release Sub-Team

Primary Design Focus

- GitLab repository setup and structure
- Branching strategy (GitFlow, trunk-based, etc.)
- Build/run instructions and automation
- README documentation structure
- CI/CD pipeline design
- Deployment workflow
- Environment management (dev, staging, prod)

Key Deliverables

- Class diagram showing build/deployment system components
- Use cases: build project, deploy to environment, merge code, etc.
- State machine: build/deployment pipeline states
- Planning: repo structure, CI/CD tools, documentation standards

Additional Tasks

- Create comprehensive README template
- Set up GitLab project structure
- Configure CI/CD pipelines
- Define merge request process
- Establish code review standards
- Document local development setup

Milestones

- Setup and maintain project README file with instructions on how to build the project and use it
- Get started with CI/CD pipeline and integrated automated tests to ensure continuous, smooth, and bug-free integration
- Setup build scripts
- Merge changes from different sub-teams seamlessly

Dependencies

- Test scripts from the Quality + Testing sub-team which will get integrated into the CI pipeline
- Commits from all sub-teams
- Release versioning rules

High-Level Task Breakdown

- Define git strategy (trunk-based vs. GitFlow)
- Configure merge request approval requirements
- Establish commit message conventions
- Upkeep the README
- Create branch protection rules

6 Submission Checklist

Required Files Per Sub-Team

- `class_diagram_[subteam].png` or `class_diagram_[subteam].svg`
- `use_cases_[subteam].pdf`
- `state_machine_[subteam].png` or `state_machine_[subteam].svg`
- `planning_[subteam].pdf`

Submission Process

- Verify all files are named correctly
- Verify all files are in correct format
- Submit all files to D2L before deadline

- Confirm submission was successful

7 Individual Requirements

Worklog on GitLab Issue Board

- Log minimum 3 hours per week
- **Critical:** Do not log all hours on due date
- Distribute work entries throughout the week
- Include specific task descriptions
- Be honest about time spent

GitLab Issue Board Worklog Format

- Task name
- Time spent
- Date completed
- Brief description of work done

Peer Evaluation

- Complete peer evaluation via ITP Metrics
- Submit before deadline
- Provide constructive feedback
- Be honest and fair in assessments

Presentation Preparedness

- Understand all aspects of your sub-team's design
- Be able to explain your personal contributions
- Be able to answer questions about:
  - Your class diagram
  - Your use cases
  - Your state machine
  - Your planning document
  - Integration with other sub-teams

8 Key Reminders & Principles

Platform-First Thinking

Focus on shared platform systems. Design for reusability across different games. Don't just think about specific game logic. Build abstractions that work for multiple scenarios.

Consistency Across Sub-Teams

Coordinate naming conventions, align on interface definitions, use consistent diagram notation, and cross-validate designs with dependent teams.

Stubs & Integration Points

Identify where database connections will reside, where server/API connections will be made, define clear interface contracts, and document stub implementations needed for P2.

Communication Requirements

Hold regular sync meetings with dependent sub-teams, document integration point agreements, escalate conflicts/blockers early, and share design updates proactively.

9 Project Milestones (Overall)

P2 Phase (Current – Design)

- Complete all design deliverables
- Present design in tutorial
- Submit all required documents
- Validate design with instructors

P3 Phase (Implementation)

- Implement designed systems
- Integration testing
- End-to-end testing
- Final demonstration
- Project documentation

10 Timeline Recommendations

Week 1

- Initial sub-team meeting
- Assign roles and responsibilities
- Begin class diagram
- Identify use cases
- Start GitLab Issue Board logging

Week 2

- Complete class diagram draft
- Complete use case descriptions
- Begin state machine diagram

- Coordinate with dependent teams
- Continue GitLab Issue Board logging

Week 3

- Finalize all diagrams
- Complete planning document
- Prepare presentation
- Review all deliverables
- Practice presentation
- Continue GitLab Issue Board logging

Week 4 (Submission Week)

- Final review of all documents
- Submit to D2L
- Deliver presentation
- Complete peer evaluation
- Final GitLab Issue Board updates

11 Quality Checklist

Class Diagram

- All classes have attributes and methods
- Relationships are clearly labeled
- Interface points to other teams are marked
- UML notation is correct
- Diagram is readable and well-organized

Use Case Diagram + Descriptions

- 2–4 use cases identified
- All actors shown
- System boundary clear
- Each use case has complete description
- All sections (actors, preconditions, main flow, postconditions, exceptions) included

State Machine Diagram

- Initial and final states marked
- All transitions labeled
- All states reachable
- No orphaned states
- Diagram represents actual system flow

Planning Document

- P2 milestones listed
- P3 milestones listed
- Dependencies clearly identified
- Task breakdown is realistic
- Document is well-organized

Presentation

- Covers all required topics
- All team members prepared
- Timing is appropriate
- Visuals are clear
- Q&A preparation complete

12 Tools & Resources

Recommended Diagramming Tools

- UML diagrams: Lucidchart, Draw.io, PlantUML, StarUML
- State machines: Draw.io, Lucidchart, Visual Paradigm
- Mockups (UI team): Figma, Sketch, Adobe XD

Document Creation

- PDF generation: Microsoft Word, Google Docs, LaTeX
- Diagrams: export from diagramming tools

Collaboration

- GitLab Issue Board: worklog tracking
- GitLab: code repository (Integration team to set up)
- Shared documents: Google Drive, OneDrive

13 Common Pitfalls to Avoid

- Don't wait until the due date to log GitLab Issue Board hours
- Don't work in isolation – communicate with other teams
- Don't create overly game-specific designs
- Don't forget to show integration points in diagrams
- Don't submit incorrect file formats
- Don't skip practice for the presentation
- Don't assume other teams know what you need – document it

14 Success Criteria

Your sub-team's design is successful if:

- All deliverables are complete and submitted on time
- Designs are coherent and well-thought-out
- Integration points with other teams are clear
- All team members understand the design
- Presentation demonstrates deep understanding
- Individual worklogs show consistent effort
- Dependencies are identified and communicated
- Design is platform-first and reusable